

Phase 2 Structured Notes: 2-3-4 Trees

0. Big Idea of Phase 2

Phase 2 teaches **2-3-4 trees**.

A 2-3-4 tree is a balanced search tree where one node can store **more than one key**.

In Phase 1, you learned about **2-3 trees**.

A 2-3 tree node can store:

- 1 key, or
- 2 keys

A 2-3-4 tree is wider.

A 2-3-4 tree node can store:

- 1 key,
- 2 keys, or
- 3 keys

So the main idea is:

A 2-3-4 tree is like a 2-3 tree, but each node is allowed to hold one extra key.

This makes the tree wider and shorter.

A shorter tree usually means faster searching, insertion, and deletion.

1. Current Progress Before Phase 2

Before Phase 2, you should already understand these ideas from Phase 1:

1.1 Binary Search Tree idea

A Binary Search Tree, or BST, stores one key per node.

Example:

```
    30
   /
  20  40
```

The rule is:

```
smaller values go left
larger values go right
```

1.2 Why a normal BST can become slow

If we insert sorted data into a normal BST:

```
10, 20, 30, 40, 50
```

The tree may become a chain:

```
10
 20
 30
 40
 50
```

This is bad because searching may take **$O(n)$** time.

1.3 Multi-way search tree idea

A multi-way search tree lets one node have more than two children.

That means one node can create more than two search directions.

Example:

```
[30, 50]
```

This creates three directions:

values less than 30
values between 30 and 50
values greater than 50

1.4 2-3 tree idea

A 2-3 tree is a balanced multi-way search tree.

A 2-3 tree node can be:

2-node: 1 key, 2 children
3-node: 2 keys, 3 children

The key Phase 1 rules were:

- All leaves stay at the same level.
- Insert into a leaf.
- If a node overflows, split it.
- Deletion must not leave a node empty.
- Borrow first.
- Merge second.

Phase 2 keeps these same ideas, but uses wider nodes.

2. What Is a 2-3-4 Tree?

A **2-3-4 tree** is a balanced multi-way search tree.

It is called a 2-3-4 tree because each internal node can behave like one of these:

Node Type	Number of Keys	Number of Child Ranges	Simple Meaning
2-node	1 key	2 child ranges	Like a normal BST node
3-node	2 keys	3 child ranges	Makes 3 search choices
4-node	3 keys	4 child ranges	Makes 4 search choices

A 2-3-4 tree is also a form of a **B-tree of degree 4**.

In simple words:

A 2-3-4 tree is a balanced tree where each node can store up to 3 keys.

3. Why Phase 2 Matters

Phase 2 is important because 2-3-4 trees are directly connected to **Red-Black Trees**.

Later, in Phase 3 and Phase 4, Red-Black Trees will be easier to understand if you already understand 2-3-4 trees.

The learning path is:

```
BST
↓
2-3 Tree
↓
2-3-4 Tree
↓
Red-Black Tree
```

The key idea is:

A Red-Black Tree is basically a 2-3-4 tree drawn in binary-tree form.

So if you understand Phase 2 well, Red-Black Trees will feel much less confusing.

4. Degree 4

4.1 What does degree mean?

The **degree** of a multi-way tree means:

the maximum number of children a node can have

A 2-3-4 tree has **degree 4**.

That means:

```
maximum children = 4
```

4.2 Why does degree 4 mean 3 keys?

Keys separate child ranges.

If a node has 4 children, it needs 3 keys to separate those 4 child ranges.

So:

```
maximum children = 4  
maximum keys = 3
```

Example node:

```
[20, 40, 60]
```

This node has 3 keys.

Those 3 keys create 4 child ranges:

```
child[0]: values less than 20  
child[1]: values greater than 20 and less than 40  
child[2]: values greater than 40 and less than 60  
child[3]: values greater than 60
```

So a 4-node can make four decisions at one node.

5. 2-Node, 3-Node, and 4-Node

5.1 2-node

A 2-node has:

```
1 key  
2 child ranges
```

Example:

[30]

Search rule:

values less than 30 go left
values greater than 30 go right

This is similar to a normal BST node.

5.2 3-node

A 3-node has:

2 keys
3 child ranges

Example:

[30, 50]

Search rule:

values less than 30 go left
values between 30 and 50 go middle
values greater than 50 go right

5.3 4-node

A 4-node has:

3 keys
4 child ranges

Example:

[20, 40, 60]

Search rule:

```
values less than 20 go to child[0]
values between 20 and 40 go to child[1]
values between 40 and 60 go to child[2]
values greater than 60 go to child[3]
```

A 4-node is full.

That means it already has the maximum number of keys allowed in a normal 2-3-4 node.

6. Main Rules of a 2-3-4 Tree

A 2-3-4 tree follows these rules:

1. A node may store 1, 2, or 3 keys.
2. A node with 1 key has up to 2 children.
3. A node with 2 keys has up to 3 children.
4. A node with 3 keys has up to 4 children.
5. Keys inside a node must always be sorted.
6. Duplicate keys are not allowed.
7. All leaves must stay at the same level.
8. Insertion happens at a leaf.
9. A node with 4 keys is only temporary.
10. If a node temporarily gets 4 keys, it must split.
11. Deletion must never leave a node empty.
12. If deletion creates an empty node, borrow first and merge second.

The most important balance rule is:

All leaves must stay at the same level.

This rule keeps the tree balanced.

7. Comparison: 2-3 Tree vs 2-3-4 Tree

Feature	2-3 Tree	2-3-4 Tree
Degree	3	4

Feature	2-3 Tree	2-3-4 Tree
Maximum children per node	3	4
Maximum keys per node	2	3
Node types	2-node, 3-node	2-node, 3-node, 4-node
Overflow happens at	3 temporary keys	4 temporary keys
Balance rule	All leaves same level	All leaves same level
Deletion repair	Borrow, then merge	Borrow, then merge

Simple summary:

A 2-3-4 tree is a wider version of a 2-3 tree.

8. Node Structure in C++

A 2-3-4 tree node can be represented like this:

```
struct Node {
    int keys[3];           // Can hold up to 3 keys
    Node* child[4];        // Can point to up to 4 children
    int keyCount;          // Current number of keys: 1, 2, or 3
    bool isLeaf;           // True if the node has no children

    Node() {
        keyCount = 0;
        isLeaf = true;
        for (int i = 0; i < 4; i++) {
            child[i] = nullptr;
        }
    }
};
```

8.1 Meaning of each field

`keys[3]`

This stores the keys inside the node.

A 2-3-4 node can store at most 3 keys.

Example:

```
keys[0] = 20  
keys[1] = 40  
keys[2] = 60
```

This represents:

```
[20, 40, 60]
```

`child[4]`

This stores pointers to child nodes.

A 2-3-4 node can have at most 4 children.

For this node:

```
[20, 40, 60]
```

The children mean:

```
child[0]: values < 20  
child[1]: 20 < values < 40  
child[2]: 40 < values < 60  
child[3]: values > 60
```

`keyCount`

This tells how many keys are currently inside the node.

Possible normal values are:

```
1, 2, or 3
```

A temporary overflow may create 4 keys during insertion, but a real 2-3-4 node cannot permanently have 4 keys.

`isLeaf`

This tells whether the node is a leaf.

A leaf is a node with no children.

Insertion usually happens at a leaf.

`nullptr`

A child pointer is set to `nullptr` when that child does not exist.

Example:

```
child[i] = nullptr;
```

This means there is no child at that position.

9. Searching in a 2-3-4 Tree

Searching is like searching in a BST, but each node may have more than one key.

Example node:

```
[20, 40, 60]
```

Search for 10

```
10 < 20  
Go to child[0]
```

Search for 30

```
20 < 30 < 40  
Go to child[1]
```

Search for 50

```
40 < 50 < 60  
Go to child[2]
```

Search for 90

```
90 > 60  
Go to child[3]
```

Beginner search rule

1. Check whether the target is inside the current node.
2. If found, stop.
3. If not found, choose the correct child range.
4. Repeat until found or until there is no child to follow.

10. Balance Rule

The most important balance rule is:

All leaves must be at the same level.

10.1 Valid tree

```
      [40]  
     /  
    [20]  [60, 80]  
   /  \  /  |  
 [10] [30] [50] [70] [90,100]
```

All leaves are on the same level.

This is valid.

10.2 Invalid tree

```
      [40]  
     /  
    [20]  [60]  
   /  \        
 [10] [30]    [80]  
                  
              [90]
```

The right side is deeper than the left side.

This is invalid.

10.3 Why this rule matters

If all leaves stay level, the tree height stays small.

That is why search, insert, and delete stay fast.

11. Insertion in a 2-3-4 Tree

Insertion begins like BST insertion.

First, search downward to find the correct leaf.

Then insert the new key into that leaf.

There are two main cases:

Case 1: The leaf has space.

Case 2: The leaf is full and must split.

12. Insertion Case 1: Node Has Space

A 2-3-4 node can hold up to 3 keys.

So if the leaf has 1 or 2 keys, we can insert directly.

Example:

Insert 10:
[10]

Then insert 20:

[10, 20]

Then insert 30:

```
[10, 20, 30]
```

This is still valid because a 2-3-4 node can hold 3 keys.

No split is needed yet.

13. Insertion Case 2: Node Is Full

A node with 3 keys is full.

Example:

```
[10, 20, 30]
```

Now insert 40.

Temporary overflow:

```
[10, 20, 30, 40]
```

A real 2-3-4 node cannot permanently hold 4 keys.

So we split.

14. Right-Biased Split

The Phase 2 notes use a **right-biased split**.

Suppose a node temporarily has 4 sorted keys:

```
[a, b, c, d]
```

The right-biased split is:

```
left node keeps: [a]
key moves up:    b
right node gets: [c, d]
```

This is called **right-biased** because the right node gets 2 keys while the left node gets 1 key.

14.1 Example split

Insert 40 into:

```
[10, 20, 30]
```

Temporary overflow:

```
[10, 20, 30, 40]
```

Right-biased split:

```
    [20]
   /
[10]  [30, 40]
```

Meaning:

```
10 stays in the left node
20 moves up
30 and 40 go into the right node
```

14.2 Important beginner note

Do not think the split is always perfectly even.

In this version, the split is right-biased.

```
[a, b, c, d] becomes [a], b moves up, [c, d]
```

15. Conceptual Split Function

The split idea can be described like this:

```
void splitNode(Node* parent, Node* fullNode) {
    // 1. We have 4 keys total: 3 old keys plus 1 new key.
```

```
// 2. Sort the 4 keys.  
// 3. Right-biased split:  
//   - 1 key stays in the left node.  
//   - 2 keys go to a new right node.  
//   - 1 middle key moves UP to the parent.  
// 4. If the parent is also full, split the parent too.  
}
```

In simple words:

When a node becomes too full, split it into two nodes and move one key upward.

16. Insertion Dry Run: 10 Through 100

Now let us insert:

10, 20, 30, 40, 50, 60, 70, 80, 90, 100

This dry run shows normal insertion, splitting, and cascading splitting.

Step 1: Insert 10

The tree is empty, so 10 becomes the root.

[10]

Step 2: Insert 20

The root has space.

[10, 20]

Step 3: Insert 30

The root still has space.

[10, 20, 30]

Now the root is full, but it is still valid.

Step 4: Insert 40

The root is full.

Temporary overflow:

[10, 20, 30, 40]

Right-biased split:

 [20]
 /
[10] [30, 40]

20 moves up and becomes the root.

Step 5: Insert 50

50 is greater than 20, so go right.

Right leaf is:

[30, 40]

It has space, so insert 50 there.

 [20]
 /
[10] [30, 40, 50]

Step 6: Insert 60

60 is greater than 20, so go right.

Right leaf is full:

```
[30, 40, 50]
```

Insert 60 temporarily:

```
[30, 40, 50, 60]
```

Right-biased split:

```
30 stays left  
40 moves up  
50 and 60 go right
```

Root receives 40.

Tree becomes:

```
      [20, 40]  
     /   |  
  [10] [30] [50, 60]
```

Step 7: Insert 70

70 is greater than 40, so go to the rightmost child.

Rightmost child is:

```
[50, 60]
```

It has space, so insert 70.

```
      [20, 40]
     /   |
    [10] [30] [50, 60, 70]
```

Step 8: Insert 80

80 goes to the rightmost child:

```
[50, 60, 70]
```

That child is full.

Temporary overflow:

```
[50, 60, 70, 80]
```

Right-biased split:

```
50 stays left
60 moves up
70 and 80 go right
```

Root receives 60.

Tree becomes:

```
      [20, 40, 60]
     /   |   |
    [10] [30] [50] [70, 80]
```

Now the root is full.

Step 9: Insert 90

90 goes to the rightmost leaf:

[70, 80]

It has space, so insert 90.

```
      [20, 40, 60]
     /   |   |
    [10] [30] [50] [70, 80, 90]
```

Step 10: Insert 100

100 goes to the rightmost leaf:

[70, 80, 90]

This leaf is full.

Temporary overflow:

[70, 80, 90, 100]

Right-biased split:

70 stays left
80 moves up
90 and 100 go right

The parent/root receives 80.

But the root is already full:

[20, 40, 60]

If it receives 80, it temporarily becomes:

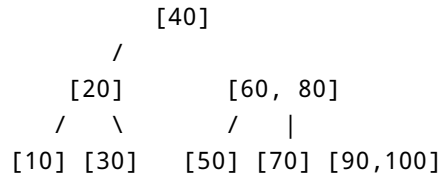
[20, 40, 60, 80]

So the root also splits.

Right-biased root split:

20 stays left
40 moves up
60 and 80 go right

Final tree:



This is a cascading split.

17. Cascading Split

A **cascading split** happens when a split pushes a key into a parent that is already full.

Example from the dry run:

Before inserting 100, the root is:

[20, 40, 60]

It is already full.

When the leaf split sends 80 up, the root becomes temporarily:

[20, 40, 60, 80]

That is not allowed.

So the root must split too.

Beginner rule:

A split can move upward.
If the parent overflows, split the parent too.

18. Deletion in a 2-3-4 Tree

Deletion is harder than insertion because a node must not become empty.

The main deletion rule is:

Never leave a node empty.

If deletion creates an empty node, fix it using this priority:

1. Borrow first.
2. Merge second.

This is the same idea as Phase 1.

19. When Can a Sibling Lend a Key?

In a 2-3-4 tree, a sibling can lend a key if it has extra keys.

A sibling has extra keys if it has:

2 keys or 3 keys

Because after lending one key, it will still have at least one key.

A sibling with only 1 key cannot lend.

20. Borrowing

Borrowing happens when a sibling has extra keys and can help fix an empty node.

20.1 Borrow from right sibling

Example:

```
    [80]
   /
  []    [90, 100]
```

The left child is empty.

The right sibling has two keys, so it can lend.

Borrowing steps:

1. Parent key 80 moves down into the empty child.
2. Right sibling key 90 moves up to the parent.
3. Right sibling keeps 100.

Result:

```
    [90]
   /
 [80]  [100]
```

20.2 Borrow from left sibling

Borrowing from the left sibling is similar.

The idea is:

1. Parent key moves down into the empty child.
2. Left sibling sends one key up to the parent.
3. Left sibling loses one key but remains valid.

20.3 Why borrowing is preferred

Borrowing is preferred because it fixes the empty node without shrinking the tree.

It keeps the tree structure more stable.

21. Merging

Merging happens when borrowing is not possible.

Borrowing is not possible when the sibling has only one key.

Example:

```
    [20]
   /
 [10]  []
```

The right child is empty.

The left sibling [10] has only one key, so it cannot lend.

So we merge:

```
[10] + parent key 20 + empty node
```

Result:

```
[10, 20]
```

21.1 Important note about merging

Merging pulls a key down from the parent.

So the parent loses one key.

If the parent becomes empty, the repair may continue upward.

This means deletion repairs can cascade upward, just like insertion splits can cascade upward.

22. Conceptual Deletion Repair Function

The deletion repair idea can be described like this:

```
void fixNode(Node* parent, int index) {
    // Priority 1: Borrow from Right Sibling
    if (rightSiblingHasExtraKeys()) {
        borrowRight();
        return;
    }
```

```
// Priority 2: Borrow from Left Sibling
if (leftSiblingHasExtraKeys()) {
    borrowLeft();
    return;
}

// Priority 3: Merge with a Sibling
mergeNodes();
}
```

In simple words:

First try the right sibling.
Then try the left sibling.
If nobody can lend, merge.

23. Deletion Dry Run Cases

The Phase 2 material discusses several deletion cases.

Case 1: Delete 100

Suppose the leaf is:

[90, 100]

Delete 100:

[90]

This is simple because the node still has one key.

No borrowing or merging is needed.

Rule shown:

If a leaf still has at least one key after deletion, it is valid.

Case 2: Delete 70

Suppose deleting 70 leaves an empty node.

If a sibling has extra keys, we borrow through the parent.

Example idea:

```
Parent key: 80
Empty node: []
Right sibling: [90, 100]
```

Borrowing:

```
80 moves down into the empty node
90 moves up into the parent
sibling keeps 100
```

Result idea:

```
    [90]
   /
[80] [100]
```

Rule shown:

```
Borrowing fills the empty node by rotating keys through the parent.
```

Case 3: Delete 30

Suppose deleting 30 leaves an empty node and borrowing is not possible.

Example idea:

```
Sibling: [10]
Parent key: 20
Empty node: []
```

Merge:

```
[10] + 20 + []
```

Result:

```
[10, 20]
```

But the parent loses key 20.

If the parent becomes empty, the repair may continue upward.

Rule shown:

```
Merging can cause cascading repair.
```

Case 4: Delete 80

Suppose deleting 80 creates an empty node and borrowing is impossible.

Then we merge.

If the parent becomes empty and also cannot borrow, the merge may continue upward.

This can shrink the overall height of the tree.

Rule shown:

```
Cascading merge can reduce the height of the tree.
```

24. Internal Deletion

An internal key is a key stored in a node that has children.

Example:

```
      [40]
     /
[20]   [60, 80]
```

Here, 40 is an internal key because it is in the root and the root has children.

24.1 Do not directly remove an internal key

If you remove an internal key directly, you may break the search order.

So use this rule:

Replace the internal key with its predecessor or successor.
Then delete that replacement key from a leaf.

24.2 Predecessor

The predecessor is:

the largest key smaller than the key being deleted

24.3 Successor

The successor is:

the smallest key larger than the key being deleted

24.4 Why this works

The predecessor or successor keeps the search order correct.

After replacing the internal key, the actual physical deletion happens at a leaf or near the bottom of the tree.

That is easier to repair.

25. Complexity

A 2-3-4 tree stays balanced because all leaves stay at the same level.

That means the height of the tree stays small.

So the main operations are efficient.

Operation	Time Complexity	Simple Meaning
Search	$O(\log n)$	We travel down the height of the tree
Insert	$O(\log n)$	We search down, then may split upward
Delete	$O(\log n)$	We search down, then may repair upward
Storage	$O(n)$	We store all keys in nodes

25.1 Why operations are $O(\log n)$

The tree is balanced.

So the height grows slowly as the number of keys grows.

Local actions like split, borrow, and merge are small operations because each node has a fixed maximum size.

The main cost comes from moving down or up the height of the tree.

26. Why 2-3-4 Trees Are Useful

2-3-4 trees are useful because they are balanced and wide.

A wider tree has fewer levels.

Fewer levels means fewer steps during search.

This idea is important for databases and file systems because related tree structures, like B-trees and B+ trees, are used for indexing.

The basic reason is:

A node with multiple keys gives more information at once.

So the tree can be flatter.

27. Connection to Red-Black Trees

A major reason Phase 2 matters is that 2-3-4 trees are closely related to Red-Black Trees.

Simple idea:

A Red-Black Tree is a binary-tree version of a 2-3-4 tree.

In a 2-3-4 tree, one node may hold multiple keys.

In a Red-Black Tree, each physical node holds one key, but colors help represent grouped keys.

Later:

2-3-4 node with 1 key → black node
2-3-4 node with 2 keys → black node with one red child
2-3-4 node with 3 keys → black node with two red children

So Phase 2 prepares you for Red-Black Trees.

28. Complete Teaching Program for Phase 2

This program is a teaching demo.

It does not implement a full production 2-3-4 tree.

Instead, it demonstrates the main ideas:

- right-biased split
- borrowing
- merging
- deletion repair concepts

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;

struct Node234 {
    vector<int> keys; // Teaching version of keys[3]
```

```

    explicit Node234(initializer_list<int> init = {}) : keys(init) {
        sort(keys.begin(), keys.end());
    }
};

void printVec(const string& label, const vector<int>& v) {
    cout << label << " [";
    for (size_t i = 0; i < v.size(); ++i) {
        if (i) cout << ", ";
        cout << v[i];
    }
    cout << "]\n";
}

void rightBiasedSplit(vector<int> keys) {
    sort(keys.begin(), keys.end());

    cout << "Overflow keys: ";
    for (int x : keys) cout << x << " ";
    cout << "\n";

    vector<int> left = {keys[0]};
    int up = keys[1];
    vector<int> right = {keys[2], keys[3]};

    printVec("Left node keeps", left);
    cout << "Key moving UP: " << up << "\n";
    printVec("Right node gets", right);
    cout << "\n";
}

void borrowViaParent(Node234& empty, int& parentKey, Node234& sibling) {
    cout << "Borrow via parent\n";
    empty.keys.push_back(parentKey);
    parentKey = sibling.keys.front();
    sibling.keys.erase(sibling.keys.begin());
}

void mergeViaParent(Node234& sibling, int parentKey) {
    cout << "Merge via parent: pull " << parentKey << " down\n";
    sibling.keys.push_back(parentKey);
    sort(sibling.keys.begin(), sibling.keys.end());
}

int main() {
    cout << "Phase 2: 2-3-4 Tree Teaching Demo\n\n";
}

```

```

    cout << "Insert 40 into full root [10, 20, 30]\n";
    rightBiasedSplit({10, 20, 30, 40});

    cout << "Insert 60 into full child [30, 40, 50]\n";
    rightBiasedSplit({30, 40, 50, 60});

    cout << "Insert 100 into full leaf [70, 80, 90]\n";
    rightBiasedSplit({70, 80, 90, 100});

    cout << "If the root is also full, split the root too.\n\n";

    cout << "Delete 100 from [90, 100]: simple leaf deletion -> [90]\n\n";

    cout << "Borrow demo: empty child, parent key 80, sibling [90, 100]\n";
    Node234 empty;
    int parent = 80;
    Node234 sibling{90, 100};

    borrowViaParent(empty, parent, sibling);

    printVec("Filled child", empty.keys);
    cout << "Parent key becomes " << parent << "\n";
    printVec("Sibling", sibling.keys);
    cout << "\n";

    cout << "Merge demo: node empty, sibling [10], parent key 20\n";
    Node234 leftSibling{10};
    mergeViaParent(leftSibling, 20);
    printVec("Merged node", leftSibling.keys);

    return 0;
}

```

29. What the Program Teaches

Node234

This is a simple teaching node.

It uses a `vector<int>` instead of fixed arrays so the demo is easier to read.

A real 2-3-4 node would usually use:

```
int keys[3];  
Node* child[4];
```

`rightBiasedSplit`

This shows the split rule:

```
[a, b, c, d]
```

becomes:

```
left node: [a]  
key moves up: b  
right node: [c, d]
```

`borrowViaParent`

This shows borrowing:

```
parent key moves down  
sibling key moves up
```

`mergeViaParent`

This shows merging:

```
parent key moves down  
sibling combines with that key
```

30. Common Beginner Mistakes

Mistake 1: Confusing degree with number of keys

Wrong idea:

```
Degree 4 means 4 keys.
```

Correct idea:

Degree 4 means 4 children.
A node with 4 children has 3 keys.

Mistake 2: Thinking a 4-node has 4 keys

Wrong:

A 4-node has 4 keys.

Correct:

A 4-node has 3 keys and 4 child ranges.

Example:

[20, 40, 60]

This is a 4-node.

Mistake 3: Letting a node permanently hold 4 keys

A 2-3-4 node can hold at most 3 keys.

Four keys are only temporary during insertion.

Wrong permanent node:

[10, 20, 30, 40]

Correct after split:

 [20]
 /
[10] [30, 40]

Mistake 4: Forgetting keys inside a node must stay sorted

Wrong:

[60, 20, 40]

Correct:

[20, 40, 60]

If the keys are not sorted, we cannot choose the correct child range.

Mistake 5: Assuming every split is perfectly even

In this Phase 2 material, the split is right-biased.

With four keys:

[a, b, c, d]

The result is:

[a], b moves up, [c, d]

The right side gets two keys.

Mistake 6: Merging before borrowing

The correct deletion repair order is:

1. Borrow first.
2. Merge second.

Do not merge if borrowing is possible.

Mistake 7: Ignoring child pointers during internal repair

When borrowing or merging inside the tree, keys are not the only things that move.

Child pointers may also need to move.

For beginner diagrams, we focus on keys.

In real code, we must also reassign children correctly.

Mistake 8: Forgetting cascading splits

A split can push a key into the parent.

If the parent is full, the parent splits too.

This is called a cascading split.

Mistake 9: Forgetting cascading deletion repair

A merge can make the parent lose a key.

If the parent becomes empty, deletion repair may continue upward.

Mistake 10: Forgetting height shrink

If the root becomes empty after merging, the tree may shrink by one level.

This is allowed.

The tree remains valid as long as all leaves stay at the same level.

31. Key Vocabulary

2-3-4 tree

A balanced multi-way search tree where each node can store 1, 2, or 3 keys.

Degree

The maximum number of children a node can have.

Degree 4

A node can have at most 4 children.

2-node

A node with 1 key and 2 child ranges.

3-node

A node with 2 keys and 3 child ranges.

4-node

A node with 3 keys and 4 child ranges.

Key range

The group of values that belongs to a child pointer.

Example:

```
[20, 40, 60]
```

creates these ranges:

```
< 20  
20 to 40  
40 to 60  
> 60
```

Leaf

A node with no children.

Root

The top node of the tree.

Split

Breaking an overflowing node into two nodes and moving one key upward.

Right-biased split

A split where:

[a, b, c, d] becomes [a], b moves up, [c, d]

Cascading split

A split that causes the parent to overflow, forcing another split upward.

Borrow

A deletion repair where a sibling gives one key through the parent.

Merge

A deletion repair where an empty node, a parent key, and a sibling combine.

Predecessor

The largest key smaller than a given key.

Successor

The smallest key larger than a given key.

$O(\log n)$

A fast time complexity where the number of steps grows slowly as input grows.

32. Phase 2 Final Summary

A 2-3-4 tree is a balanced multi-way search tree.

It is wider than a 2-3 tree.

A 2-3-4 tree node can be:

2-node: 1 key, 2 child ranges
3-node: 2 keys, 3 child ranges
4-node: 3 keys, 4 child ranges

The tree has degree 4, meaning:

```
maximum children = 4  
maximum keys = 3
```

A node like this:

```
[20, 40, 60]
```

creates four ranges:

```
values < 20  
20 < values < 40  
40 < values < 60  
values > 60
```

The most important balance rule is:

```
All leaves must stay at the same level.
```

Insertion happens at a leaf.

If a node temporarily gets 4 keys, it splits.

The Phase 2 split is right-biased:

```
[a, b, c, d] → [a], b moves up, [c, d]
```

Deletion must never leave a node empty.

If deletion creates an empty node:

1. Borrow first.
2. Merge second.

Because the tree stays balanced:

```
Search =  $O(\log n)$   
Insert =  $O(\log n)$ 
```

Delete = $O(\log n)$
Storage = $O(n)$

Final one-sentence memory:

A 2-3-4 tree is a balanced search tree where each node can hold up to 3 keys, splits when a 4th key appears, and fixes deletion by borrowing or merging.

33. Mini Practice Questions

Question 1

What is the maximum number of keys in a 2-3-4 tree node?

Answer:

3 keys

Reason:

Degree 4 means at most 4 children, so at most 3 keys.

Question 2

What is a 4-node?

Answer:

A node with 3 keys and 4 child ranges.

Example:

[20, 40, 60]

Question 3

In this node:

```
[20, 40, 60]
```

Where does 50 go?

Answer:

```
child[2]
```

Reason:

```
40 < 50 < 60
```

Question 4

Insert 40 into:

```
[10, 20, 30]
```

What happens?

Answer:

Temporary overflow:

```
[10, 20, 30, 40]
```

Right-biased split:

```
    [20]
   /
 [10]  [30, 40]
```

Question 5

What does right-biased split mean?

Answer:

With [a, b, c, d], the split becomes:
left node [a]
b moves up
right node [c, d]

Question 6

A node becomes empty after deletion. Its sibling has 2 keys. What should we do?

Answer:

Borrow.

Reason:

Borrow before merging.

Question 7

A node becomes empty after deletion. Its sibling has only 1 key. What should we do?

Answer:

Merge.

Reason:

The sibling cannot lend because it has no extra key.

Question 8

Why are 2-3-4 trees fast?

Answer:

They stay balanced because all leaves stay at the same level.

So operations are:

$O(\log n)$

Question 9

Why does Phase 2 help with Red-Black Trees?

Answer:

Because Red-Black Trees are binary-tree versions of 2-3-4 trees.

34. One-Page Memory Sheet

2-3-4 Tree = balanced multi-way search tree

Degree 4 means:

- maximum 4 children
- maximum 3 keys

Node types:

- 2-node = 1 key, 2 child ranges
- 3-node = 2 keys, 3 child ranges
- 4-node = 3 keys, 4 child ranges

Example:

[20, 40, 60]

Ranges:

child[0] = values < 20

child[1] = 20 < values < 40

```
child[2] = 40 < values < 60  
child[3] = values > 60
```

Important rules:

- Keys inside a node stay sorted.
- No duplicate keys.
- All leaves stay at the same level.
- Insert into leaves.
- 4 keys means temporary overflow.
- Overflow causes split.
- Phase 2 uses right-biased split.
- [a,b,c,d] becomes [a], b moves up, [c,d].
- Deletion cannot leave empty nodes.
- Borrow first.
- Merge second.
- Internal deletion uses predecessor or successor.

Complexity:

Search = $O(\log n)$

Insert = $O(\log n)$

Delete = $O(\log n)$

Storage = $O(n)$

Connection:

2-3-4 trees prepare you for Red-Black Trees.

End of Phase 2 Notes.